

MedinaElizabeth_PracticaSeminarioLaLiga

July 26, 2026

1 Algoritmos de optimización - Seminario

Nombre y Apellidos: *Elizabeth Medina Toapanta*

Url: https://github.com/gemedinat/MIAR_ALGORITMOS_DE_OPTIMIZACION.git

Google Colab: https://drive.google.com/file/d/1Mk_fidADAPG8BuJP_EXIbIH7sk0bK9zz/view?usp=sharing

Problema: El problema seleccionado es el de los partidos de la liga de fútbol:

Problema 2. Organizar los horarios de partidos de La Liga(IV)

Desde la La Liga de fútbol profesional se pretende organizar los horarios de los partidos de liga de cada jornada. Se conocen algunos datos que nos deben llevar a diseñar un algoritmo que realice la asignación de los partidos a los horarios de forma que maximice la audiencia.

- Los horarios disponibles se conocen a priori y son los siguientes:

Dias	Horario
Viernes	20
Sábado	12,16,18,20
Lunes	20

- En primer lugar se clasifican los equipos en tres categorías según el numero de seguidores(que tiene relación directa con la audiencia). Hay 3 equipos en la categoría A, 11 equipos de categoría B y 6 equipos de categoría C.
- Se conoce estadísticamente la audiencia que genera cada partido según los equipos que se enfrentan y en horario de sábado a las 20h (el mejor en todos los casos):

	Cat. A	Cat. B	Cat. C
Cat. A	2 Millones	1.3 Millones	1.0 Millones
Cat. B		0.9 Millones	0.75 Millones
Cat. C			0.47 Millones

- Si el horario del partido no se realiza a las 20 horas del sábado se sabe que se reduce según los coeficientes de la siguiente tabla
- Debemos asignar obligatoriamente siempre un partido el viernes y un partido el lunes:

	Viernes	Sábado	Domingo	Lunes
12h	-	0.55	0.45	-
16h	-	0.70	0.75	-
18h	-	0.80	0.85	-
20h	0.4	1.00	1.00	0.4

- Es posible la coincidencia de horarios pero en este caso la audiencia de cada partido se verá afectada y se estima que se reduce en porcentaje según la siguiente tabla dependiendo del número de coincidencias:

Coincidencias	Porcentaje
0	0%
1	25%
2	45%
3	60%
4	70%
5	75%
6	78%
7	80%
8	80%

- Los cálculos asociados a una jornada de ejemplo se realizan según se muestra en la siguiente tabla:

Partido	Categorías Horario		Base (Mill.)	Ponderación	Base × Ponderación	Corrección Coincidencia
Celta - Real Madrid	B-A	V20	1,3	0,4	0,52	0,52
Valencia - R. Sociedad	B-A	S12	1,3	0,55	0,72	0,72
Mallorca - Eibar	C-C	S16	0,47	0,7	0,33	0,33
Athletic - Barcelona	B-A	S18	1,3	0,8	1,04	1,04
Leganés - Osasuna	C-C	S20	0,47	1	0,47	0,47
Villarreal - Granada	B-C	D16	0,75	0,75	0,56	0,42
Alavés - Levante	B-B	D16	0,9	0,75	0,68	0,51
Espanyol - Sevilla	B-B	D18	0,9	0,85	0,77	0,77
Betis - Valladolid	B-C	D20	0,75	1	0,75	0,75
Atlético - Getafe	B-B	L20	0,9	0,4	0,36	0,36

Total: 5,88 millones

Los dos partidos en **negrita** coinciden en D16 (1 coincidencia cada uno $\rightarrow -25\%$): $0,56 \times 0,75 = 0,42$ y $0,68 \times 0,75 = 0,51$.

```
[4]: # Definimos las librerías que utilizaremos en el notebook
import itertools
import random
from collections import Counter

[5]: # Definimos una lista con las franjas horarias
franjaHoraria = ['Viernes_20', 'Sabado_12', 'Sabado_16', 'Sabado_18',
                 'Sabado_20', 'Domingo_12', 'Domingo_16',
                 'Domingo_18', 'Domingo_20', 'Lunes_20']

[6]: # Creamos un diccionario con las ponderaciones de las franjas horarias
# Coeficiente de ponderación de cada franja respecto al sábado a las 20h
ponderacionFranjas = {'Viernes_20': 0.4, 'Sabado_12': 0.55, 'Sabado_16': 0.7,
                        'Sabado_18': 0.8, 'Sabado_20': 1.0,
                        'Domingo_12': 0.45, 'Domingo_16': 0.75, 'Domingo_18': 0.85,
                        'Domingo_20': 1.0, 'Lunes_20': 0.4}

[7]: # Otro dato que nos da el ejercicio es la reducción porcentual de audiencia
     # según el número de coincidencias
     # dentro de la misma franja horaria para ello creamos un diccionario.

reduccionFranjas = {0: 0.00, 1: 0.25, 2: 0.45, 3: 0.60, 4: 0.70,
                     5: 0.75, 6: 0.78, 7: 0.80, 8: 0.80}

[8]: # Dentro de las RESTRICCIONES tenemos las de estas dos franjas, por lo tanto
     # creamos un índice para cada una de ellas.
idxViernes = franjaHoraria.index('Viernes_20')
idxLunes   = franjaHoraria.index('Lunes_20')

[52]: # Audiencia en los diferentes horarios expresada en millones
# Se guarda con la tupla ordenada alfabéticamente para que (B,A) y (A,B) sean
     # equivalentes
audBase = {('A', 'A'): 2.0, ('A', 'B'): 1.3, ('A', 'C'): 1.0,
           ('B', 'B'): 0.9, ('B', 'C'): 0.75, ('C', 'C'): 0.47}

[53]: ### Función que devuelve la audiencia entre dos partidos
def audienciaBase(c1, c2):
    return audBase[tuple(sorted((c1, c2)))]
```

1.1 (*) ¿Cuántas posibilidades hay sin tener en cuenta las restricciones?

Respuesta:

El problema considera que cada fin de semana vamos a tener un cronograma de 10 partidos y lo que

estamos buscando es que cada partido se juegue en un horario donde la distribución permita que se maximice la audiencia en TV. Cuando se menciona las posibilidades sin restricciones estamos considerando que cualquiera de esos 10 partidos puede estar ubicado en cualquier franja horaria de los fines de semana y a su vez repetirse, bajo ese escenario tendríamos una permutación con los siguientes componentes: - n : número de partidos - m : número de franjas horarias Por lo tanto, el cálculo del espacio de soluciones son las aplicaciones de un conjunto de 10 elementos en otro de 10 elementos, es decir:

$$m^n = 10^{10} = 10$$

mil millones de posibilidades.

1.2 ¿Cuántas posibilidades hay teniendo en cuenta todas las restricciones?

Respuesta: Considerando las restricciones somos conscientes que algunas de las combinaciones del cálculo anterior no son viables. Si analizamos de forma intuitiva y que en algunos papers se denomina como criterio de inclusión - exclusión tendríamos las siguientes causas: - Total: tenemos el cálculo anterior

$$10^{10}$$

retiramos las franjas donde no jugaría nadie. Considerando que el viernes no se puede jugar cada partido solo tendría 9 opciones

$$9^{10}$$

(combinaciones que no se deberían considerar) - De la misma manera se quitarían las que no tienen partido el lunes que sería el mismo cálculo y deberían ser retiradas. - A pesar que se retiran las jornadas del lunes y viernes, en el cálculo se les trae de vuelta a las combinaciones restantes quedando 8^{10}

$$\begin{aligned} & 10^{10} - 2 \cdot 9^{10} + 8^{10} \\ &= 4.100.173.022 \cdot 10^{10} - 2 \cdot 9^{10} + 8^{10} \\ &= 4.100.173.022 \cdot 10^{10} - 2 \cdot 9^{10} + 8^{10} \\ &= 4.100.173.022 \end{aligned}$$

Sería el resultado de opciones válidas (41% aprox.) respecto al cálculo anterior. Pero el valor es inmenso y tiene un crecimiento de forma exponencial por lo que, podríamos considerar que utilizar un algoritmo de fuerza bruta podría no ser una opción viable.

```
[2]: #Implementación en código
#1. Declaramos nuestras dos condiciones
m = 10 # número de franjas horarias
n = 10 # número de partidos que se juega en una jornada
#2. Escenario donde cada partido puede elegirse en cualquier franja (Sin
    restricciones)
sr = m**n
print(f'Número de posibilidades sin considerar restricciones:{sr:,}')
```

Número de posibilidades sin considerar restricciones:10,000,000,000

```
[3]: #3. Escenario donde consideramos los no juegos del viernes y lunes (con
    ↪restricciones)
cr = m**n-2*(m-1)**n+(m-2)**n
print(f'Número de posibilidades considerando restricciones:{cr:,}')
print(f'Porcentaje de posibilidades considerando restricciones dado el Total:
    ↪{(cr/sr)*100:.2f}%')
```

Número de posibilidades considerando restricciones:4,100,173,022

Porcentaje de posibilidades considerando restricciones dado el Total:41.00%

1.3 Modelo para el espacio de soluciones

1.4 (*) ¿Cuál es la estructura de datos que mejor se adapta al problema? Argumenta la respuesta

Para este problema se elige una **estructura de lista** que tiene una longitud n que se encuentra en una posición i , es decir, representa al partido _{i} y su respectivo valor en la franja horaria correspondiente. Entre los aspectos revisados se encuentran:

- **Espacio de soluciones:** representa el espacio de soluciones mediante una lista que contiene n valores en $\{0, \dots, m-1\}$ donde se encuentran posibles opciones candidatas y toda solución candidata se representa de forma única.
- **Acceso y modificación en $O(1)$:** permite cambiar la franja horaria de un partido que generalmente es común en los algoritmos de búsqueda local es de forma inmediata.
- **Evaluación eficiente:** es necesario saber cuantos encuentros se dan en un horario para el calculo de la audiencia. Para ello, se utiliza una tabla hash sobre la lista se obtiene $O(n)$.
- **Compatibilidad:** como se realiza una representación vectorial es muy compatible con operadores de otros algoritmos por el estándar en metaheurísticas como cruces genéticos, entre otros.

Cuando se piensa en el plantamiento del ejercicio en un inicio se piensa en usar un **diccionario** para aprovechar la asociación de clave (horas de las franjas) - valor (partidos). No obstante, cuando empezamos a utilizar los operadores y se tiene la necesidad de actualizar las listas se pueden visualizar inconsistencias como partidos en dos franjas y no se ve las ventajas del coste. Por lo tanto, se utiliza una representación vectorial y se mantiene al diccionario como ayuda auxiliar calculada al evaluar. Los datos estáticos del problema (ponderaciones, audiencias base, reducciones) se almacenan en **diccionarios** por legibilidad y acceso $O(1)$, y cada partido se representa como una **tupla de categorías** ('B', 'A') por ser inmutable y ligera.

```
[33]: # Vamos a representar una jornada con una solución para una mejor comprensión
# Una jornada = lista de 10 partidos; cada partido = tupla (categoría_local,
    ↪categoría_visitante)

ejemploFranjas = [
    ('B', 'A'), # Celta- Real Madrid
    ('B', 'A'), # Valencia- R. Sociedad
    ('C', 'C'), # Mallorca- Eibar
    ('B', 'A'), # Athletic- Barcelona
```

```

('C', 'C'), # Leganés- Osasuna
('B', 'C'), # Villarreal- Granada
('B', 'B'), # Alavés- Levante
('B', 'B'), # Espanyol- Sevilla
('B', 'C'), # Betis- Valladolid
('B', 'B'), # Atlético- Getafe
]

```

```

[34]: # Una solución = lista de índices de franja (posición i -> franja del partido i)
solucion = [franjaHoraria.index(h) for h in
            ['Viernes_20', 'Sabado_12', 'Sabado_16', 'Sabado_18',
            ↪ 'Sabado_20', 'Domingo_16',
            'Domingo_16', 'Domingo_18', 'Domingo_20', 'Lunes_20']]

```

```

[35]: print("Vector Solución:", solucion)
      print("Franjas Partidos:", [franjaHoraria[h] for h in solucion])

```

Vector Solución: [0, 1, 2, 3, 4, 6, 6, 7, 8, 9]

Franjas Partidos: ['Viernes_20', 'Sabado_12', 'Sabado_16', 'Sabado_18',
'Sabado_20', 'Domingo_16', 'Domingo_16', 'Domingo_18', 'Domingo_20', 'Lunes_20']

1.5 Según el modelo para el espacio de soluciones

1.6 (*) ¿Cuál es la función objetivo?

Sea $S = (s_1, \dots, s_n)$ una asignación de franjas, b_i la audiencia base del partido i (según categorías), $p(s_i)$ la ponderación de la franja asignada y $c_i = |\{j \neq i : s_j = s_i\}|$ el número de coincidencias del partido i en su franja. La audiencia total de la jornada es:

$$f(S) = \sum_{i=1}^n b_i \cdot p(s_i) \cdot (1 - r(c_i))$$

donde $r(\cdot)$ es la tabla de reducciones por coincidencia. La función está sujeta a la restricción de factibilidad: $\exists i : s_i = V20$ y $\exists j : s_j = L20$.

1.7 (*) ¿Es un problema de maximización o minimización?

En este caso, estamos tratando con un problema de **MAXIMIZACIÓN**, el ejercicio busca conseguir la mayor audiencia total de espectadores (millones). Representado matemáticamente sería:

$$S^* = \arg \max f(S)$$

Para verificar el cálculo se contrasta con lo mencionado en el ejercicio, cuyo resultado debe ser aprox. **5,88 millones**.

```

[54]: # Como primer paso, se define una función que permita calcular la asignación de
      ↪ la franja del partido_i y partidos que
      # se representarán a modo de tupla según cada categoría.

```

```
def audienciaTotal(franH, partidos):
    ocupacion = Counter(franH) # n° de partidos por franja -> O(n)
    total = 0.0
    for i, franja in enumerate(franH):
        c1, c2 = partidos[i]
        coincidencias = ocupacion[franja] - 1 # partidos que comparten
        ↪franja horaria
        factor = 1 - reduccionFranjas[min(coincidencias, 8)] # la tabla satura
        ↪en 80% (8) por coincidencias
        total += audienciaBase(c1, c2) *
        ↪ponderacionFranjas[franjaHoraria[franja]] * factor
    return total
```

```
[55]: # Definimos una segunda función para validar las restricciones >=1 partido
        ↪viernes y >=1 lunes
```

```
def validador(franH):
    return idxViernes in franH and idxLunes in franH
```

```
[58]: valor = audienciaTotal(solucion, ejemploFranjas) # Llamamos el set de datos
        ↪para aplicar la función
```

```
print(f"* La audiencia de la jornada utilizada en el ejemplo: {valor:.2f}
        ↪millones (esperado planteamiento: 5.88 mill)")
assert round(valor, 2) == 5.88, "El modelo no reproduce el ejemplo del
        ↪enunciado"
print("* El modelo fue validado adecuadamente")
```

```
* La audiencia de la jornada utilizada en el ejemplo: 5.88 millones (esperado
planteamiento: 5.88 mill)
```

```
* El modelo fue validado adecuadamente
```

1.8 Diseña un algoritmo para resolver el problema por fuerza bruta

Aplicar un algoritmo de fuerza, en un simil consite en aplicar un producto cartesiano ya que, enumeramos las m^n asignaciones que pueden ser posibles en el espacio de soluciones con la finalidad de descartar aquellas que no son adecuadas y en este caso quedarnos con aquella que nos genera mayor audiencia asegurando el **óptimo global** por que recorre todo el espacio de soluciones.

Para este problema, como se ha mencionada en ítems anteriores se tiene un 10^{10} exploraciones o evaluaciones que demorarían más de un día si consideramos como ejemplo 10^6 evaluaciones/segundo (tardaría apróx. 28 horas) y el costo de cada recorrido generaría un coste de $O(n)$.

También considero importante mencionar que la aplicación de la fuerza bruta sirve para entender el problema, verificar el óptimo cuando resolvemos problemas en pequeña escala que nos sirve para validar la metaheurística y manifestar la necesidad de la aplicación de otros algoritmos.

```
[ ]: # Aplicación del Algoritmo de Búsqueda Exhaustiva
```

```
[74]: # Aquí tenemos todos los partidos que pueden jugarse en los diferentes horarios
      ↪(debemos considerar todos sin restricciones)
def fuerzaBruta (partidos, idxFranja):
    mValor, mSolucion = -1.0, None
    evaluadas = 0
    # itertools: m^n combinaciones
    for sol in itertools.product(idxFranja, repeat=len(partidos)):
        # Aseguramos al menos un partido el viernes y otro el lunes
        if idxViernes not in sol or idxLunes not in sol:
            continue
        evaluadas += 1
        valor = audienciaTotal(list(sol), partidos) # Ocupamos la función del
        ↪enunciado anterior
        if valor > mValor:
            mValor, mSolucion = valor, list(sol)
    return mValor, mSolucion, evaluadas
```

```
[68]: # Dado que aplicar el algoritmo en toda su expresión demoraría mucho y consume
      ↪muchos recursos
# Se establece una simulación a 5 partidos con 5 franjas
francasDemostracion = [franjaHoraria.index(h) for h in ['Viernes_20',
        ↪'Sabado_20',
                                                    'Domingo_18',
        ↪'Domingo_20',
                                                    'Lunes_20']]
partidosDemostracion= [('A', 'A'), ('A', 'B'), ('B', 'B'), ('B', 'C'), ('C',
        ↪'C')]
```

```
[75]: import time # Esta librería permite medir cuanto se demora la ejecución del
      ↪algoritmo
t0 = time.time()
valor, sol, evaluadas = fuerzaBruta(partidosDemostracion, francasDemostracion)
t1 = time.time()
```

```
[79]: # Impresión de resultados
print(f"Espacio total explorado para la demostración : 5^5 = {5**5:,}
      ↪asignaciones ({evaluadas:,} factibles)")
print(f"Óptimo global: {valor:.4f} millones")
print(f"Asignación óptima : {[franjaHoraria[h] for h in sol]}")
print(f"Tiempo de ejecución: {t1 - t0:.3f} s")
```

Espacio total explorado para la demostración : 5⁵ = 3,125 asignaciones (1,320 factibles)

Óptimo global: 4.5530 millones

Asignación óptima : ['Sabado_20', 'Domingo_20', 'Domingo_18', 'Viernes_20', 'Lunes_20']

Tiempo de ejecución: 0.018 s

1.9 Calcula la complejidad del algoritmo por fuerza bruta

La complejidad del algoritmo representa un crecimiento **exponencial** en función del número de partidos que se generen. Se puede descomponer de la siguiente manera:

- m^n : producto cartesiano completo o número de soluciones generadas dentro del espacio.
- $O(n)$: representa la función objetivo que va recorriendo los n partidos y construye el contador de ocupación (costo de evaluar cada solución).

$$T(n) = (m^n \cdot n)$$

Para la instancia real $n = m$ se generan 10^{11} operaciones elementales del orden del día u horas considerando un escenario convencional. Si duplicamos las dos entradas se convierte en una dificultad alarmante ya que en este tipo de algoritmos no se escala. La complejidad espacial, es solo $O(n)$ donde guardamos la solución actual y la mejor que se ha encontrado.

```
[ ]: # Simulación del crecimiento exponencial del algoritmo de fuerza bruta
    ↪(escenarios)
```

```
[84]: # Generamos un bucle que nos muestre el tamaño creciente y lo extrapolamos a n
    ↪= 10
for j in [1, 3, 5, 6]:
    franjaSim = [franjaHoraria.index('Viernes_20'), franjaHoraria.
    ↪index('Lunes_20')] + \
                [franjaHoraria.index(h) for h in ['Sabado_20', 'Domingo_20',
    ↪'Sabado_18', 'Domingo_18'][:j-2]]
    partidoSim = [('A', 'B')] * j
    t0 = time.time()
    fuerzaBruta(partidoSim, franjaSim) # Usamos la función del apartado anterior
    t = time.time() - t0
    print(f"n = m = {j}: {j**j:>8,} asignaciones -> {t:8.4f} s")
```

```
n = m = 1:      1 asignaciones -> 0.0000 s
n = m = 3:     27 asignaciones -> 0.0000 s
n = m = 5:    3,125 asignaciones -> 0.0124 s
n = m = 6:   46,656 asignaciones -> 0.1764 s
```

```
[86]: # Extrapolación con el último tiempo medido (coste ~ n * m^n)
costoUnitario = t / (6 * 6**6)
estimado = costoUnitario * 10 * 10**10
print(f"\Considerado a n = m = 10: ~{estimado / 3600:.1f} horas -> No
    ↪disponible con los recursos actuales")
```

```
\Considerado a n = m = 10: ~17.5 horas -> No disponible con los recursos
actuales
```

1.10 (*) Diseña un algoritmo que mejore la complejidad del algoritmo por fuerza bruta. Argumenta por qué crees que mejora el algoritmo por fuerza bruta

Búsqueda Local (Hill Climbing) con multiarranque aleatorio : esta técnica busca una buena solución la mejora poco a poco, toma una solución en cualquier punto y en cada paso evalúa (mira) a los vecinos (generan algunos cambios) y a medida que avanza se queda con la que mejora en este caso la audiencia. Es un proceso que se repite hasta cuando ya no existe ningún cambio por más pequeño que sea hasta el punto donde obtiene un óptimo local.

Para este ejercicio funciona de la siguiente manera: - Genera una solución factible aleatorio garantizando un partido el Viernes_20 y Lunes_20. - Explora cada vecino de la solución elegida, en este caso todas las que difieren en la franja del único partido sería $(n \cdot (m-1)) = 90$ vecinos y siempre va a saltar cuando se de una mejora en la audiencia. - Se genera un multiarranque para que el hill climbing no quede atorado en óptimos locales hasta tener x soluciones iniciales aleatorias distintas y se obtenga la mejor.

Respecto al algoritmo de **Fuerza Bruta** utilizado anteriormente ya no se recorre los 10^{10} puntos del espacio ya que cada escalada solo evalúa en este caso el tamaño de 90 vecindades. Asimismo, se aprovecha de soluciones que tienen audiencias parecidas moviéndose siempre hacia donde hay mejora.

```
[ ]: # Construimos las funciones para el algoritmo de búsqueda local Hill Climbing
    ↪ con Multiarranque
```

```
[87]: def solucionFactibleAleatoria(numPartidos):
        """Genera una asignación aleatoria que cumple la restricción V20/L20."""
        solucion = [random.randrange(len(franjaHoraria)) for _ in
        ↪ range(numPartidos)]
        i, j = random.sample(range(numPartidos), 2) # dos partidos distintos
        solucion[i] = idxViernes                    # Se fuerza un partido para
        ↪ el viernes
        solucion[j] = idxLunes                      # Se fuerza un partido para
        ↪ el lunes
        return solucion
```

```
[102]: # En este paso se mejora la solución haciendo saltos al mejor vecino (n*(m-1)
        ↪ vecinos) hasta tener el óptimo local
def hillClimbing(partidos, solucion):
    mValor = audienciaTotal(solucion, partidos)
    evaluaciones = 1
    mejora = True
    while mejora:
        mejora = False
        mejorVecino, mVecinoValor = None, mValor
        for i in range(len(solucion)):                # ejecución para
        ↪ cada partido
            franjaOriginal = solucion[i]
            for h in range(len(franjaHoraria)):        # se prueba para
        ↪ cada una de las franjas
```

```

        if h == franjaOriginal:
            continue
        solucion[i] = h                                # generamos el
↪vecino in-place
        if validador(solucion):
            valor = audienciaTotal(solucion, partidos)
            evaluaciones += 1
            if valor > mVecinoValor + 1e-12:
                mVecinoValor = valor
                mejorVecino = (i, h)
            solucion[i] = franjaOriginal                # se restauran las
↪franjas originales
        if mejorVecino is not None:                    # saltamos al mejor vecino
            i, h = mejorVecino
            solucion[i] = h
            mValor = mVecinoValor
            mejora = True
        return mValor, solucion, evaluaciones

```

[96]: *# Definimos una función que ejecute reinicios desde puntos aleatorios para*
↪devolver el mejor

```

def busquedaMultiarranque(partidos, reinicios=100, semilla=42):
    random.seed(semilla)                                # aseguramos la
↪reproducibilidad
    mValor, mSolucion, totalEvaluacion = -1.0, None, 0
    for _ in range(reinicios):
        valor, solucion, evaluacion = hillClimbing(partidos,
↪solucionFactibleAleatoria(len(partidos)))
        totalEvaluacion += evaluacion
        if valor > mValor:
            mValor, mSolucion = valor, solucion[:]
    return mValor, mSolucion, totalEvaluacion

```

[113]: `optFuerzaBruta, _, _ = fuerzaBruta(partidoSim, franjaSim)`
`optBusqMulti, _, _ = busquedaMultiarranque(partidoSim, reinicios=50)`
`print(f"Óptimo fuerza bruta (reducida): {optFuerzaBruta:.4f}")`
`print(f"Óptimo búsqueda local multiarranque: {optBusqMulti:.4f} {'*IGUALA EL`
`↪ÓPTIMO GLOBAL' if abs(optFuerzaBruta-optBusqMulti) < 1e-9 else ''}")`

Óptimo fuerza bruta (reducida): 5.7850

Óptimo búsqueda local multiarranque: 5.7850 *IGUALA EL ÓPTIMO GLOBAL

[110]: *# Consideramos las 10 franjas y los 10 partidos*
`t0 = time.time()`
`valor, solucion, evaluaciones = busquedaMultiarranque(ejemploFranjas,`
`↪reinicios=200)`

```

t1 = time.time()
print(f"\n--- Resultados: Franja establecida en el ejercicio ---")
print(f"Mejor audiencia encontrada : {valor:.4f} millones (vs 5.88 de la
↪ asignación del ejemplo)")
print(f"Asignación correspondiente: {[franjaHoraria[h] for h in solucion]}")
print(f"Evaluaciones realizadas: {evaluaciones:,} (vs 10^10 de la fuerza
↪ bruta)")
print(f"Tiempo ejecución: {t1 - t0:.2f} s")

```

```

--- Resultados: Franja establecida en el ejercicio ---
Mejor audiencia encontrada : 6.8560 millones (vs 5.88 de la asignación del
ejemplo)
Asignación correspondiente: ['Sabado_20', 'Domingo_18', 'Viernes_20',
'Domingo_20', 'Lunes_20', 'Sabado_12', 'Sabado_16', 'Sabado_18', 'Domingo_12',
'Domingo_16']
Evaluaciones realizadas: 80,048 (vs 10^10 de la fuerza bruta)
Tiempo ejecución: 1.04 s

```

1.11 (*) Calcula la complejidad del algoritmo

Para este algoritmos contamos con un R que hace mención al número de reinicios, un I que son las interacciones hasta llegar al óptimo local, n los partidos y m las franjas horarias.

La **interacción** es la que va explorando la vecindad completa tomando $(n \cdot (m - 1))$ vecinos y evalúa el costo de cada uno $O(n)$ individual y $O(n^2 m)$ por cada interacción. Cada escalada realiza I interacciones y cada salto realiza una mejora.

$$T = O(R \cdot I \cdot n^2 \cdot m)$$

Considerado lo planteado tenemos lo siguiente:

($R = 200$, $I \approx 10$, $n = m = 10$): $\approx 2 \cdot 10^6$ operaciones frente a 10^{11} del algoritmo de Fuerza Bruta.

```

[114]: # Comparación del algoritmo frente al de "FUERZA BRUTA"
print(f"{'n=m':>4} | {'Fuerza Bruta (ops ~ n·m^n)':>28} | {'Búsqueda Local (ops
↪ ~ R·I·n^2·m)':>26}")
print("-" * 68)
R, I = 200, 10 # parámetros típicos observados
for j in [1, 5, 10, 15, 20]:
    fuerzaBruta = j * j**j
    busquedaLocal = R * I * j**2 * j
    print(f"{'j':>0} | {'fuerzaBruta:>28,'} | {'busquedaLocal:>26,'}")
print("\nLa fuerza bruta crece exponencialmente; la búsqueda local,
↪ polinómicamente.")

```

n=m	Fuerza Bruta (ops ~ $n \cdot m^n$)	Búsqueda Local (ops ~ $R \cdot I \cdot n^2 \cdot m$)
1	1	2,000
5	15,625	250,000
10	100,000,000,000	2,000,000
15	6,568,408,355,712,890,625	6,750,000
20	2,097,152,000,000,000,000,000,000,000	16,000,000

La fuerza bruta crece exponencialmente; la búsqueda local, polinómicamente.

1.12 Según el problema (y tenga sentido), diseña un juego de datos de entrada aleatorio

Para que sea más realista vamos a utilizar la composición del ejercicio planteado del enunciado: **3 equipos A, 11 B y 6 C**.

```
[116]: # Creamos una función para generar la jornada de partidos de forma aleatoria
def juegosAleatorios (nA=3, nB=11, nC=6, semilla=None):
    assert (nA + nB + nC) % 2 == 0, "El número total de equipos debe ser par"
    if semilla is not None:
        random.seed(semilla)
    equipos = ['A'] * nA + ['B'] * nB + ['C'] * nC # plantilla de la liga del
    ↪ejercicio
    random.shuffle(equipos) # sorteo del
    ↪emparejamiento entre equipos
    return [(equipos[i], equipos[i + 1]) for i in range(0, len(equipos), 2)]
```

```
[117]: # En el siguiente bloque generados tres ejemplos de combinaciones
for s in [1, 2, 3]:
    jornada = juegosAleatorios(semilla=s) # (con semilla, para
    ↪reproducibilidad).
    print(f"Jornada aleatoria (semilla={s}): {jornada}")
```

```
Jornada aleatoria (semilla=1): [('B', 'B'), ('C', 'C'), ('B', 'A'), ('C', 'A'),
('C', 'B'), ('B', 'B'), ('C', 'B'), ('B', 'B'), ('B', 'A'), ('C', 'B')]
Jornada aleatoria (semilla=2): [('B', 'B'), ('C', 'B'), ('C', 'C'), ('B', 'A'),
('B', 'B'), ('C', 'B'), ('B', 'B'), ('C', 'B'), ('B', 'C'), ('A', 'A')]
Jornada aleatoria (semilla=3): [('B', 'B'), ('B', 'B'), ('C', 'C'), ('A', 'B'),
('A', 'A'), ('B', 'B'), ('C', 'B'), ('C', 'B'), ('B', 'C'), ('C', 'B')]
```

1.13 Aplica el algoritmo al juego de datos generado

1.13.1 Respuesta

Aplicamos la búsqueda local con multiarranque a varias jornadas aleatorias generadas.

```
[120]: #Aplicación de las funciones anteriormente creadas para el juego de datos
↳ generado
```

```
def solucionObtenida(partidos, solucion, valor):
    orden = sorted(range(len(solucion)), key=lambda i:
↳ franjaHoraria[solucion[i]]) # Solución ordenada por franja
    for i in orden:
        c1, c2 = partidos[i]
        b = audienciaBase(c1, c2)
        p = ponderacionFranjas[franjaHoraria[solucion[i]]]
        coinc = solucion.count(solucion[i]) - 1
        f = 1 - reduccionFranjas[min(coinc, 8)]
        print(f"    {franjaHoraria[solucion[i]]}: partido {c1}-{c2} -> "
              f"{b} × {p} × {f:.2f} = {b * p * f:.3f} M")
    print(f"Total Audiencia: {valor:.4f} millones\n")
```

```
[121]: # Ejecutamos los algoritmos de búsqueda local en el set de datos
for s in [1, 2, 3]:
    jornada = juegosAleatorios(semilla=s)
    valor, solucion, _ = busquedaMultiarranque(jornada, reinicios=200,
↳ semilla=100 + s)
    print(f"*****Jornada aleatoria {s}*****")
    solucionObtenida(jornada, solucion, valor)
```

```
*****Jornada aleatoria 1*****
Domingo_12: partido C-C -> 0.47 × 0.45 × 1.00 = 0.211 M
Domingo_16: partido B-B -> 0.9 × 0.75 × 1.00 = 0.675 M
Domingo_18: partido B-B -> 0.9 × 0.85 × 1.00 = 0.765 M
Domingo_20: partido B-A -> 1.3 × 1.0 × 1.00 = 1.300 M
Lunes_20: partido C-B -> 0.75 × 0.4 × 1.00 = 0.300 M
Sabado_12: partido C-B -> 0.75 × 0.55 × 1.00 = 0.413 M
Sabado_16: partido B-B -> 0.9 × 0.7 × 1.00 = 0.630 M
Sabado_18: partido C-A -> 1.0 × 0.8 × 1.00 = 0.800 M
Sabado_20: partido B-A -> 1.3 × 1.0 × 1.00 = 1.300 M
Viernes_20: partido C-B -> 0.75 × 0.4 × 1.00 = 0.300 M
Total Audiencia: 6.6940 millones
```

```
*****Jornada aleatoria 2*****
Domingo_12: partido C-B -> 0.75 × 0.45 × 1.00 = 0.338 M
Domingo_16: partido B-B -> 0.9 × 0.75 × 1.00 = 0.675 M
Domingo_18: partido B-B -> 0.9 × 0.85 × 1.00 = 0.765 M
Domingo_20: partido B-A -> 1.3 × 1.0 × 1.00 = 1.300 M
Lunes_20: partido B-C -> 0.75 × 0.4 × 1.00 = 0.300 M
Sabado_12: partido C-B -> 0.75 × 0.55 × 1.00 = 0.413 M
Sabado_16: partido C-B -> 0.75 × 0.7 × 1.00 = 0.525 M
Sabado_18: partido B-B -> 0.9 × 0.8 × 1.00 = 0.720 M
Sabado_20: partido A-A -> 2.0 × 1.0 × 1.00 = 2.000 M
Viernes_20: partido C-C -> 0.47 × 0.4 × 1.00 = 0.188 M
Total Audiencia: 7.2230 millones
```

```

*****Jornada aleatoria 3*****
Domingo_12: partido B-C -> 0.75 × 0.45 × 1.00 = 0.338 M
Domingo_16: partido B-B -> 0.9 × 0.75 × 1.00 = 0.675 M
Domingo_18: partido B-B -> 0.9 × 0.85 × 1.00 = 0.765 M
Domingo_20: partido A-B -> 1.3 × 1.0 × 1.00 = 1.300 M
Lunes_20: partido C-B -> 0.75 × 0.4 × 1.00 = 0.300 M
Sabado_12: partido C-B -> 0.75 × 0.55 × 1.00 = 0.413 M
Sabado_16: partido C-B -> 0.75 × 0.7 × 1.00 = 0.525 M
Sabado_18: partido B-B -> 0.9 × 0.8 × 1.00 = 0.720 M
Sabado_20: partido A-A -> 2.0 × 1.0 × 1.00 = 2.000 M
Viernes_20: partido C-C -> 0.47 × 0.4 × 1.00 = 0.188 M
Total Audiencia: 7.2230 millones

```

Se puede ver que los partidos con mayor audiencia se asignan a las franjas de máxima ponderación mientras que los partidos en nivel medio tienen franjas obligatorias con audiencias bajas y evitan coincidencia por la penalización que supera casi siempre la ganancia de ocupar un mejor horario.

```

[123]: # Liga del doble de tamaño (40 equipos, 20 partidos)
# La fuerza bruta necesitaría explorar 20~20 10~26 asignaciones
# la búsqueda local lo resuelve en segundos.

ajusteJuegos = juegosAleatorios(nA=6, nB=22, nC=12, semilla=8)
t0 = time.time()
valor, solucion, evaluacion = busquedaMultiarranque(ajusteJuegos,
↳reinicios=100, semilla=7)
t1 = time.time()
print(f"Liga de 40 equipos (20 partidos, 10 franjas)")
print(f"Mejor audiencia: {valor:.4f} M | evaluaciones: {evaluacion:}, | ↳
↳tiempo: {t1 - t0:.2f} s")

```

```

Liga de 40 equipos (20 partidos, 10 franjas)
Mejor audiencia: 9.9979 M | evaluaciones: 127,864 | tiempo: 2.94 s

```

1.14 Enumera las referencias que has utilizado (si ha sido necesario) para llevar a cabo el trabajo

- Material de la asignatura *03MIAR — Algoritmos de Optimización* (VIU): videoconferencias VC1–VC4 y diapositivas del Trabajo Práctico.
- Brassard, G., y Bratley, P. (1997). *Fundamentos de algoritmia*. Prentice Hall. ISBN 13: 9788489660007.
- Duarte, A., Pantrigo, J. J., y Gallego, M. (2008). *Metaheurísticas*. Madrid: Dykinson.
- Guerequeta, R., y Vallecillo, A. (2000). *Técnicas de diseño de algoritmos*. Universidad de Málaga. <http://www.lcc.uma.es/~av/Libro/indice.html>
- Russell, S., y Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4.^a ed.), cap. 4: búsqueda local (hill climbing, reinicios aleatorios, simulated annealing).

- Documentación oficial de Python 3: módulos `itertools`, `random` y `collections`. <https://docs.python.org>
-

1.15 Describe brevemente en unas líneas cómo crees que es posible avanzar en el estudio del problema. Ten en cuenta incluso posibles variaciones del problema y/o variaciones al alza del tamaño

Algunos papers y de lo que se ha revisado en clase ante este tipo de variaciones o alzas en el tamaño podemos apostar por los siguientes puntos:

- Utilizar algoritmos con mejoras como **recocido simulado** que acepta empeoramientos con probabilidad decreciente, **búsqueda tabú** con memoria de movimientos prohibidos o **algoritmos genéticos** donde la representación vectorial se ajusta como un cromosoma. En este punto, depende mucho del camino que se quiera seguir ya que contamos con muchos algoritmos que nos pueden ayudar a sobrellevar estas variantes.
- En el caso de que se incorporen nuevas variantes por ingreso de nuevos factores ahí ya debemos analizar la inclusión de algoritmos con un enfoque más **multiobjetivo** o introducir aspectos como **incertidumbre**. Asimismo, podríamos considerar algoritmos de **Machine Learning** si tenemos historia, ciclos o generar optimización (Pareto Front). Ya depende mucho de lo que se búsque solucionar o plantear.